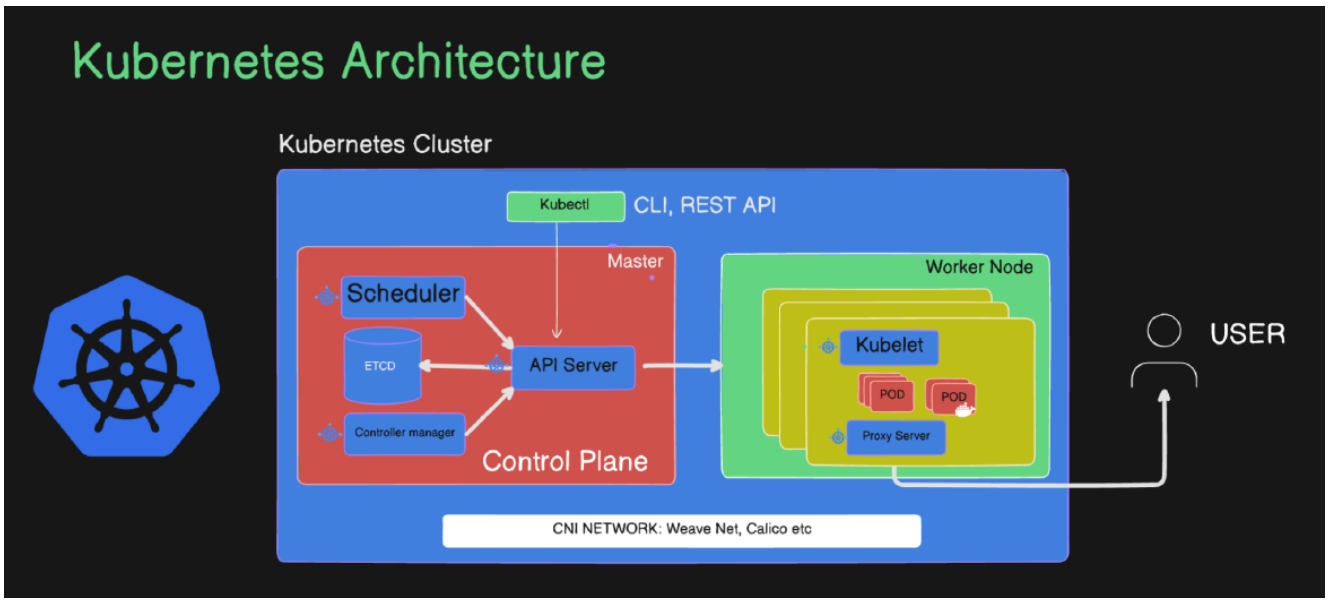
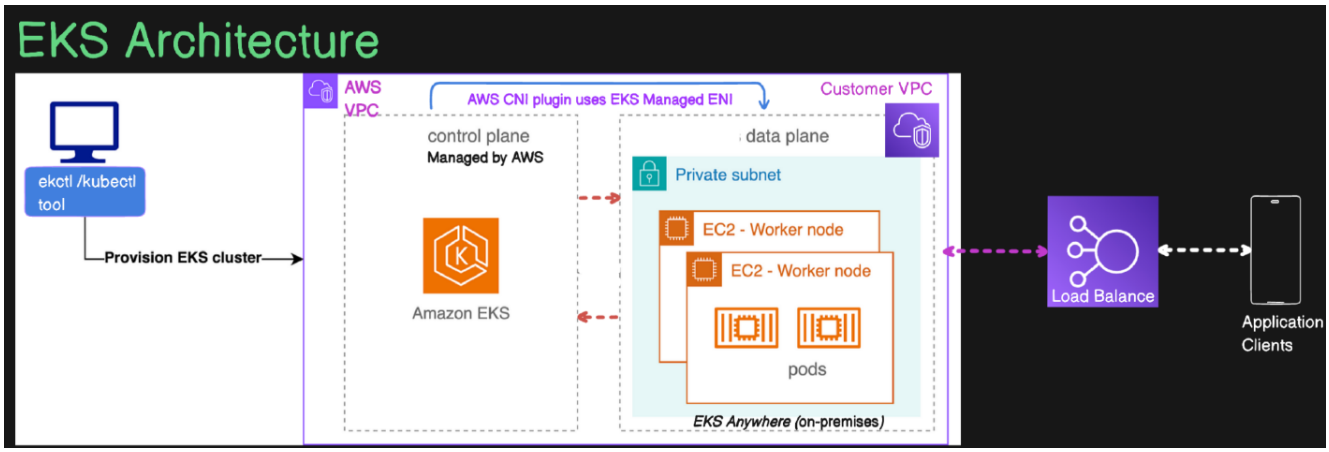
Elastic Kubernetes Service (EKS)

AWS Managed Service: AWS manages the auto-scaling and auto-healing for you. You don't have to do all the hard work, AWS will manage all these services.





AWS Fargate is a serverless compute engine for containers (similar in concept to AWS Lambda being serverless for functions). It allows you to run containers using **Amazon ECS** (or EKS) without managing

servers or clusters of Amazon EC2 instances.

With AWS Fargate, you don't need to provision, configure, or scale virtual machines. This eliminates the need to choose instance types, manage cluster capacity, or optimize resource utilization — Fargate handles it for you.

Deploy Full Stack Application on AWS EKS

3 Simple Ways to Create an EKS Cluster

1. AWS Console
2. Eksctl
3. Terraform *(by coding AWS services as infrastructure as code)*

2. Create Cluster with Eksctl

Pre-requisites: AWS knowledge

- In an EC2 machine, you will run the `eksctl` command to provide instructions and trigger EKS services
- In the same machine, you will install `kubectl` and run commands to manage pods in the cluster
- IAM to give access to the EC2 machine
- AWS CLI to run commands via SSH
- AWS Region: Try to avoid N. Virginia or Ohio

STEP 1

Create an EC2 Instance

Create an EC2 instance (t2.micro is sufficient) — this machine is *not* for the cluster itself, it is only used to trigger the EKS cluster creation.

STEP 2

Connect via SSH

Connect to the machine in your terminal via SSH, then update the system:

```
$ sudo apt update
$ sudo apt upgrade
```

STEP 3

Create IAM User with Administrator Access

Create an IAM user and attach the Administrator Access policy:

```
IAM → Users → Create User → "eks-user" → Next → Attach Policy Directly →
AdministratorAccess → Next → Create User
```

STEP 4

Install AWS CLI on the EC2 Machine

You need AWS CLI in the EC2 machine to grant IAM user access. [LINK]
Run these commands to download, unzip, and install:

```
$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
$ sudo apt install unzip
$ unzip awscliv2.zip
$ sudo ./aws/install
```

Verify the installation:

```
$ aws --version
```

You can also clean up: `$ ls -a`
`$ rm -rf awscliv2.zip`

STEP 5

Configure AWS Credentials

Associate the IAM user with this EC2 machine. First, create an Access Key from the IAM user:

```
IAM → Users → "eks-user" → Create Access Key → CLI → I understand → Next → Create Access
Key
```

```
$ aws configure
```

Note: Leave Default output format as `[None]` unless you prefer JSON/text.

STEP 6A

Install eksctl

```
$ curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz" | tar xz -C /tmp
```

```
$ sudo mv /tmp/eksctl /usr/local/bin
```

Verify:

```
$ eksctl version$ eksctl
```

> ✅ **Note:** `eksctl` is the tool used to **create** an EKS cluster.

STEP 6B

Install kubectl [LINK]

Download:

```
$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

Validate the download (you should see **OK**):

```
$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"

$ echo "$(cat kubectl.sha256)  kubectl" | sha256sum --check
```

Install kubectl:

```
$ sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Test the installed version:

```
$ kubectl version --client
```

> ✅ **Note:** `kubectl` is the tool used to **interact** with a Kubernetes cluster.

STEP 6C

Create the EKS Cluster

Choose your node type (t2.medium recommended for testing):

```
$ eksctl create cluster --name full-stack-cluster --region us-west-2 --node-type c7i-flex.large --nodes-min 2 --nodes-max 2
```

Other node type options:

```
$ eksctl create cluster --name full-stack-cluster --region us-west-2 --node-type t3.micro --nodes-min 2 --nodes-max 2

or,

$ eksctl create cluster --name full-stack-cluster --region us-west-2 --node-type t2.medium --nodes-min 2 --nodes-max 2
```

💡 This command tells EKS to create a VPC and within that, the Control Plane will be running. It triggers **AWS CloudFormation** to allocate the necessary resources.

AWS CloudFormation is a service that allows you to model, provision, and manage AWS resources using templates, treating infrastructure as code. It automates deployments for consistent, repeatable infrastructure setups.

After creation, explore your cluster:

```
$ kubectl get pods
$ kubectl get namespace
$ kubectl get pods -n kube-system
$ kubectl get nodes
$ eksctl get cluster
```

💡 **Switching between clusters:** If you have multiple clusters (e.g., `full-stack-cluster` and `nur-cluster`), point `kubectl` to the correct one:

```
$ aws eks update-kubeconfig --name full-stack-cluster --region us-west-2$ kubectl get nodes
```

should show nodes from full-stack-cluster

STEP 7

Clone the Application Repository

```
$ git clone https://github.com/NI-Shourav/K8S-Starter.git
$ cd K8S-Starter/eks
```

STEP 9

Deploy All Manifest Files

Apply all YAML files to run the application inside pods:

```
$ kubectl apply -f namespace.yaml
$ kubectl apply -f mysql-secrets.yml
$ kubectl apply -f mysql-deployment.yml
$ kubectl apply -f mysql-svc.yml
$ kubectl apply -f two-tier-app-deployment.yml
$ kubectl apply -f two-tier-app-svc.yml
```

Get service info (find the LoadBalancer external IP):

```
$ kubectl get svc -n two-tier-ns
```

Copy the **EXTERNAL-IP** of the LoadBalancer and paste it into your browser — the app is served via the load balancer.

STEP 10

Manage and Debug Pods

Get all resources in the namespace:

```
$ kubectl get all -n two-tier-ns
```

Delete all resources running inside pods:

```
$ kubectl delete -f .
```

Debug a specific pod:

```
$ kubectl describe pod/mysql-6d576469c4-67929 -n two-tier-ns
```

> ⚠ If you encounter this scheduling error: Warning FailedScheduling — 0/2 nodes are available: 2 Too many pods. No preemption victims found for incoming pod. Check the pod logs: kubectl logs mysql-98596f974-5crh5 -n two-tier-ns

Delete the Cluster

```
$ eksctl delete cluster --name <cluster_name> --region <region_name>
Example:
$ eksctl delete cluster --name full-stack-cluster --region us-west-2
```

⚠ Important Notes

- If you created any other AWS resources manually (like RDS, Load Balancers, S3 buckets, IAM roles, etc.), those **will not be deleted** automatically — you'll need to remove them manually.
- If you used a custom VPC, `eksctl` won't delete it either.

Optional Manual Cleanup Checklist

1. CloudFormation Stacks

`eksctl` uses CloudFormation under the hood. Go to the **CloudFormation Console** and delete any remaining stacks related to your cluster.

2. VPCs

Check your **VPC Dashboard** to make sure the VPC created for the cluster has been removed (if it was created by eksctl).

3. IAM Roles

Delete any cluster-specific IAM roles (optional — only if you're certain nothing else depends on them).

4. EBS Volumes & Load Balancers

Check for leftover **EBS volumes** and **ELBs** in the EC2 dashboard — they may not be auto-deleted in edge cases.